Danmarks
Tekniske
Universitet

02613 Python and HPC

Authors

Cristian Placinta - s234615
Tobias Abilock Mikkelsen - s234731
Emile Ditlefsen - s234872
Valeria Bubuioc - s215242

April 27, 2026

Task 1

Each building is provided as two NumPy arrays:

- a domain array of size 512x512, containing the initial temperature field;
- an interior mask of the same size, indicating which grid points belong to the building interior.

For visualisation, the domain and interior mask were plotted side by side. The domain plot reveals the spatial distribution of fixed-temperature boundaries, while the interior mask clearly outlines the geometry of the building.

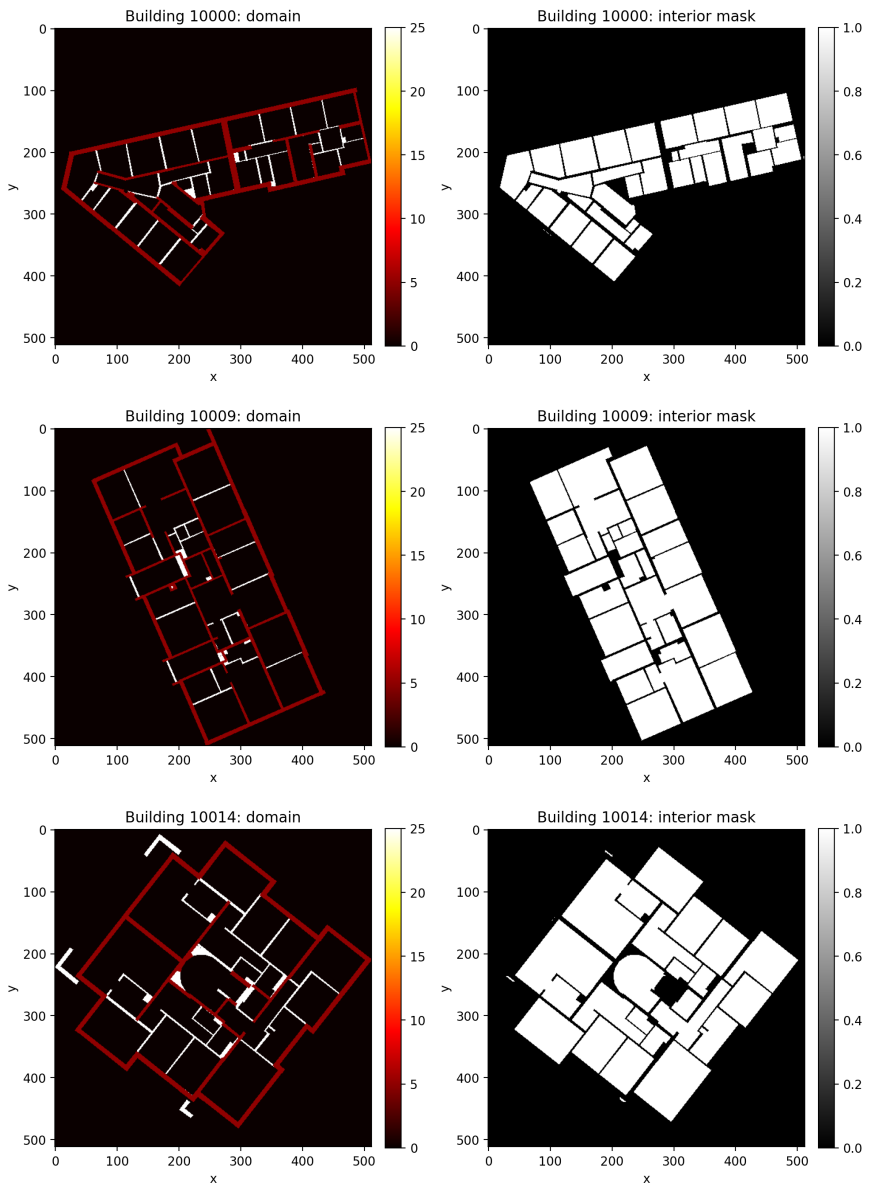

Figure 1: Example floorplans

Task 2

We timed the reference implementation as a batch job on the `hpc` queue using 1 core. For a subset of $N = 10$ floorplans, we performed 5 runs to obtain a more stable estimate of the wall-clock time. The measured runtimes were

$$114,\ 102,\ 104,\ 108,\ 90 \quad \text{(seconds)}$$

with an average of

$$T_{10} = 103.6 \text{ s}.$$

Assuming the runtime scales approximately linearly with the number of processed floorplans, the total runtime for all 4571 buildings can be estimated as

$$T_{\text{all}} \approx T_{10} \cdot \frac{4571}{10}.$$

Using the measured average, we obtain

$$T_{\text{all}} \approx 103.6 \cdot \frac{4571}{10} = 47353.56 \text{ s} \approx 13.15 \text{ h}.$$

We therefore estimate that the reference implementation would take roughly 13.2 hours to process the full dataset.

Task 3

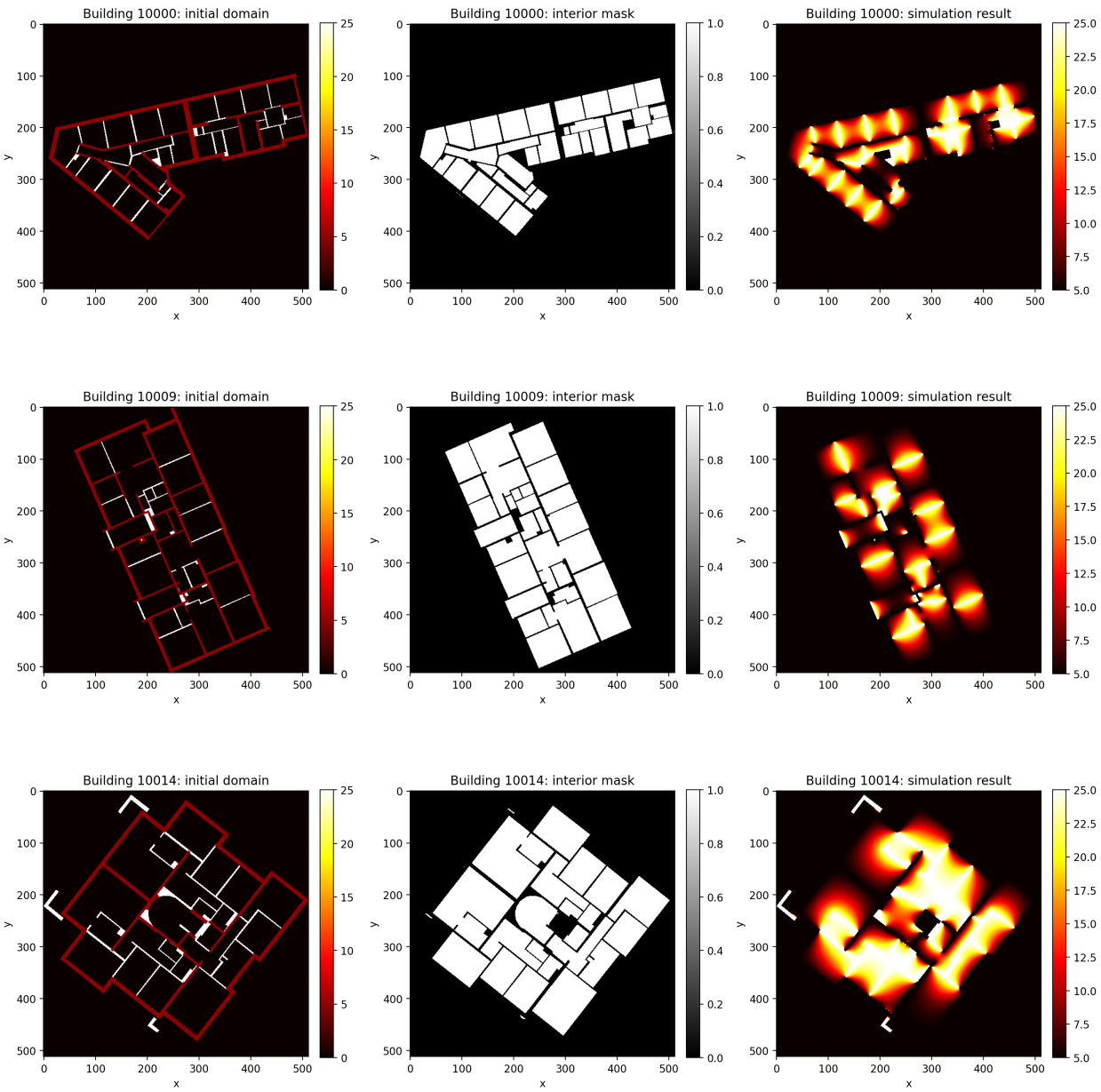

Figure 2: Simulation results for three example floorplans. Each row shows the initial domain, the interior mask, and the final temperature distribution after solving the steady-state heat equation with the Jacobi method

Task 4

We profiled the reference `jacobi` function using `kernprof` and repeated the profiling experiment 5 times in total in order to obtain a more reliable estimate. The values reported below are averages over these 5 runs.

Line 22

The first executed line inside the function is

```
u = np.copy(u)
```

This makes a copy of the input array `u`, so the function works on a local copy instead of changing the original input directly. From the profiling results, this line takes $6.56\,ms$ to run. On average, it accounts for less than 0.01% of the total runtime, so its cost is negligible.

For loop

We now move to the main loop,

```
for i in range(max_iter):
```

For the 10 floorplans used in the profiling run, this loop was hit 47282 times. This means that all lines inside the loop are repeated 47282 times, which is why the cost of the array operations quickly dominates the runtime.

Line 26

The first line in the for loop is

```
u_new = 0.25 * (u[1:-1, :-2] + u[1:-1, 2:] + u[:-2, 1:-1] + u[2:, 1:-1])
```

This is the main Jacobi update. It computes the new temperature values by averaging the four neighboring grid points. This is the most expensive line in the whole function. On average, it took 53 seconds, corresponding to 55.16% of the total runtime. The reason is simple: this line works on almost the full grid, creates temporary arrays, and is repeated 47282 times. So even though the operation is simple, it becomes the main bottleneck.

Line 27

```
u_new_interior = u_new[interior_mask]
```

This applies the boolean mask and keeps only the values at the interior points of the building. In other words, it removes walls and points outside the building, so only the room interior is kept for the update. This line took on average 10.5 seconds, corresponding to 11.10% of the total runtime. Even though the operation looks simple, it is still fairly expensive because boolean masking is not a cheap contiguous memory access. Instead, NumPy has to gather the selected values into a new array, and this is repeated 47282 times.

Line 28

```
delta = np.abs(u[1:-1, 1:-1][interior_mask] - u_new_interior).max()
```

This computes the largest change between the old and new values at the interior points. That value is stored in `delta` and is later used to check whether the method has converged.

It took on average 20.1 seconds, corresponding to 20.96% of the total runtime. It is the second most expensive line in the function. The reason is that it does several operations at once: it applies the mask again, computes the difference, takes the absolute value, and then finds the maximum. Since this is also repeated 47282 times, the total cost becomes large.

Line 29

```
u[1:-1, 1:-1][interior_mask] = u_new_interior
```

This writes the updated values back into the grid, but only at the interior points. So after computing the new temperatures, this is the line that actually updates the solution for the next Jacobi iteration.

This line took on average 11.85 seconds, corresponding to 12.62% of the total runtime. It is also fairly expensive because it uses boolean masked assignment, which is slower than a simple contiguous write. Since it is executed 47282 times, the total cost adds up.

Line 31

```
if delta < atol:
```

This checks whether the largest update is smaller than the tolerance `atol`. If that is the case, the solution is considered converged and the loop stops. This line took on average 0.09 seconds, corresponding to only 0.10% of the total runtime. So even though this line is important for stopping the iterations, its own cost is negligible. The expensive part is computing `delta`, not comparing it with `atol`.

Line 32

Moving inside the if statement we have

```
break
```

This stops the `for` loop once the convergence condition has been satisfied. Since `jacobi` was called once for each of the 10 floorplans in the profiling run, this line can be reached at most 10 times. In our case, it was reached exactly 10 times, which means that all 10 simulations stopped because they satisfied the tolerance criterion before hitting the maximum number of iterations. This line took essentially no time and its contribution to the total runtime is negligible.

Line 33

The last line of the function is

```
return u
```

This returns the final temperature grid after the Jacobi iterations have finished. This line took on average of 24 micro-seconds to run, making its cost negligible.

```
Total time: 91.9959 s
File: ../simulate.py
Function: jacobi at line 20

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    20                                           @profile
    21                                           def jacobi(u, interior_mask, max_iter, atol=1e-6):
    22        10       6435.8    643.6      0.0      u = np.copy(u)
    23
    24     47282      52361.4      1.1      0.1      for i in range(max_iter):
    25                                                   # Compute average of left, right, up and down neighbors, see eq. (1)
    26     47282   49987481.7   1057.2     54.3          u_new = 0.25 * (u[1:-1, :-2] + u[1:-1, 2:] + u[:-2, 1:-1] + u[2:, 1:-1])
    27     47282   10480436.7    221.7     11.4          u_new_interior = u_new[interior_mask]
    28     47282   19443188.9    411.2     21.1          delta = np.abs(u[1:-1, 1:-1][interior_mask] - u_new_interior).max()
    29     47282   11930024.6    252.3     13.0          u[1:-1, 1:-1][interior_mask] = u_new_interior
    30
    31     47282      95974.7      2.0      0.1          if delta < atol:
    32        10         12.5      1.3      0.0              break
    33        10         22.8      2.3      0.0      return u
```

Figure 3: Example line profiler output for the reference `jacobi` function from one of the five profiling runs.

Task 5

To speed up the reference implementation, we parallelized the computation over floorplans rather than changing the `jacobi` solver itself. More precisely, we kept the original code in `simulate.py` unchanged and built a separate script on top of it, reusing `load_data`, `jacobi`, and `summary_stats`. In this way, each worker processes a fixed subset of floorplans independently, and the final results are collected once all workers have finished.

We used static scheduling, meaning that the floorplans were divided into fixed chunks before the computation started, and each worker was assigned one chunk. Thus, there is no dynamic redistribution of work during execution. For the timing experiments, we used the first 50 floorplans and tested the worker counts

$$p \in \{1, 2, 4, 5, 10\}.$$

The timings were performed as batch jobs on the `hpc` queue for comparable results.

1. Speed-up measurement

Figure 4 shows the measured speed-up of the static scheduling implementation together with the ideal linear speed-up. The measured speed-up increases as more workers are added, so the parallelization is clearly beneficial. In particular, the speed-up grows from about 1.7 at 2 workers to about 5.2 at 10 workers.

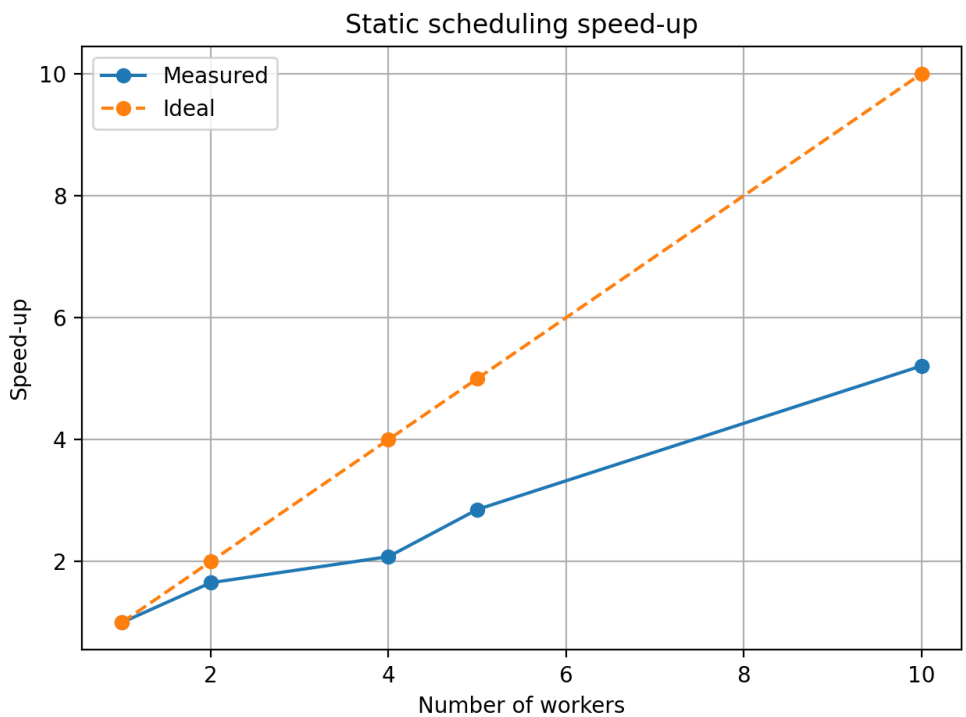

Figure 4

At the same time, the measured curve remains well below the ideal line. This shows that the implementation does not scale perfectly, mainly because of serial parts of the program and multiprocessing overhead. The difference becomes more visible for larger worker counts, where the gain from adding more workers is smaller than in the ideal case.

2. Parallel fraction estimation

To estimate how much of the computation is parallelized, we use Amdahl's law

$$S(p) = \frac{1}{(1-f) + f/p},$$

where $S(p)$ is the measured speed-up for p workers and f is the parallel fraction.

Solving for f gives

$$f = \frac{1 - \frac{1}{S(p)}}{1 - \frac{1}{p}}.$$

Using the measured speed-up at $p = 10$, where $S(10) \approx 5.2$, we get

$$f = \frac{1 - \frac{1}{5.2}}{1 - \frac{1}{10}} \approx 0.90.$$

So, roughly 90% of the computation appears to be parallelized, while the remaining 10% behaves as serial work and overhead.

3. Theoretical maximum speed-up

According to Amdahl's law, the theoretical maximum speed-up is

$$S_{\max} = \frac{1}{1-f}.$$

Using the estimated parallel fraction $f \approx 0.90$, we get

$$S_{\max} = \frac{1}{1-0.90} = 10.$$

So the theoretical maximum speed-up is about 10×.

From the measurements, the best observed speed-up was approximately 5.2×, achieved with 10 workers. This means that the implementation reached

$$\frac{5.2}{10} = 0.52,$$

that is, about 52% of the theoretical maximum speed-up. Thus, the static scheduling implementation achieved a best measured speed-up of about 5.2×, which is roughly half of the Amdahl limit, and this required 10 cores.

4. All floorplans process time estimation

Using the fastest static parallel solution, 50 floorplans were processed in approximately 134 s with 10 workers. Assuming the runtime scales approximately linearly with the number of floorplans, the total runtime for all 4571 buildings can be estimated as

$$T_{\text{all}} \approx 134 \cdot \frac{4571}{50} \approx T_{\text{all}} \approx 12250.28 \text{ s} \approx 3.40 \text{ h}$$

So, using the fastest static parallel solution, we estimate that processing the full dataset would take approximately 1.23×10^4 s, or about 3.4 hours.

Task 6

1. Did it get faster? By how much?

Yes, the dynamic scheduling version was faster for every tested worker configuration. The measured runtimes are summarized in Table 1.

Workers p	Static [s]	Dynamic [s]	Improvement [%]
1	697.33	605.22	13.2
2	422.02	329.68	21.9
4	335.27	190.29	43.2
5	244.39	168.42	31.1
10	133.90	125.26	6.5

Table 1: Measured runtimes for static and dynamic scheduling on 50 floorplans. The improvement is computed relative to the static runtime.

We see that the dynamic version is faster in all cases. The size of the improvement depends on the number of workers: it is modest for 1 and 10 workers, but much larger for intermediate worker counts. The largest improvement is seen at 4 workers, where the runtime drops from 335.27 s to 190.29 s, corresponding to about 43.2% lower runtime.

This behavior matches what we would expect. Since different floorplans require different numbers of Jacobi iterations before convergence, the workload is uneven. With static scheduling, some workers may finish early while others are still processing more expensive floorplans. Dynamic scheduling reduces this imbalance by assigning new floorplans to workers as they become free, which leads to better overall utilization and lower total runtime.

2. Did the speed-up improve or worsen?

The speed-up improved for small and medium worker counts, but worsened slightly at 10 workers. From Fig. 5, the dynamic scheduling curve lies above the static one for 2, 4, and 5 workers, which means that dynamic scheduling made better use of the available workers in this range. In particular, the improvement is most visible at 4 workers, where the dynamic version reaches a noticeably higher speed-up than the static version.

At 10 workers, however, the dynamic speed-up is slightly lower than the static one. So while dynamic scheduling still gave a lower runtime in absolute terms, its speed-up relative to its own 1-worker baseline was not better in that case.

Overall, the conclusion is that dynamic scheduling generally improved the scaling, especially for intermediate worker counts, because it reduced load imbalance between workers. The only exception was the 10-worker case, where the speed-up was slightly worse. However, since these results are based on a single timing run for each worker configuration, they should be interpreted with some caution. In particular, small differences (such as the one observed at 10 workers) may be affected by measurement noise and runtime variability, so they should not be treated as a strong general conclusion on their own.

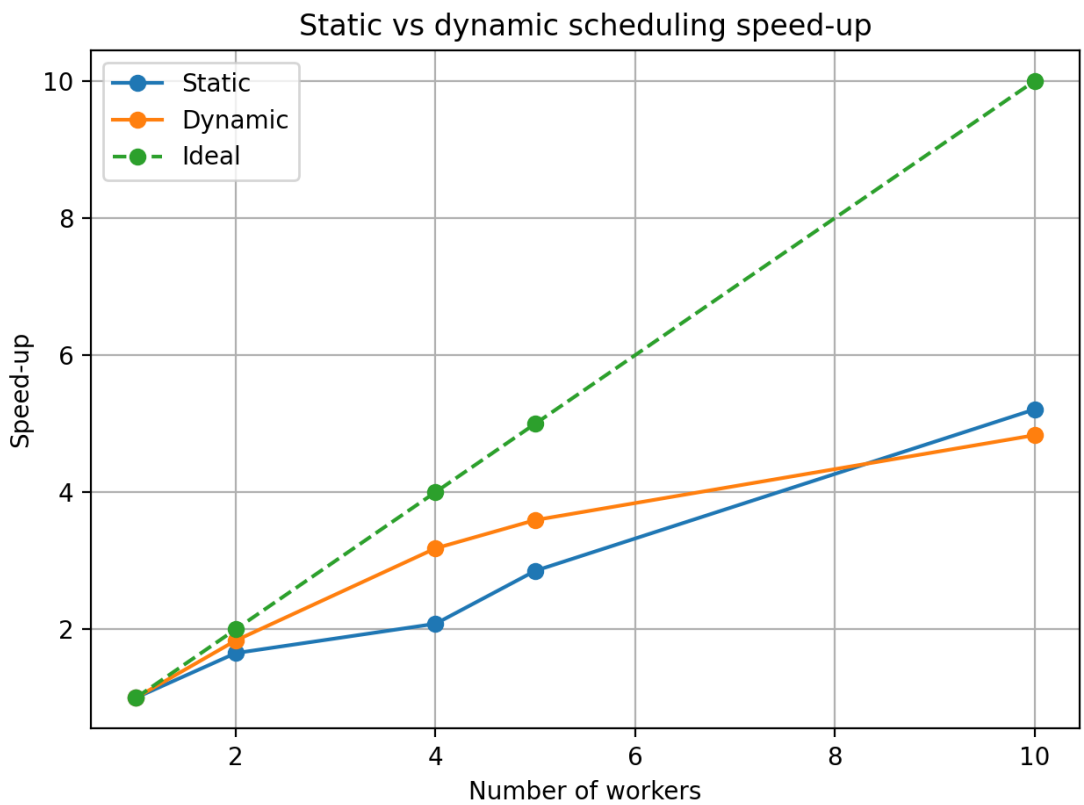

Figure 5: Measured speed-up for the static and dynamic scheduling implementations, shown together with the ideal linear speed-up.

Task 7

1. How does performance compare to the reference?

We timed the Numba JIT CPU implementation as a batch job on the `hpc` queue using 1 core. For the same subset of $N = 10$ floorplans, we again performed 5 runs in order to obtain a more stable estimate of the wall-clock time. The measured runtimes were

$$17.79,\ 16.14,\ 16.01,\ 15.94,\ 15.89 \qquad \text{(seconds)}$$

with an average of

$$T_{10,\text{Numba}} = 16.36 \text{ s}$$

For comparison, the corresponding average runtime of the reference implementation in Task 2 was

$$T_{10,\text{ref}} = 103.6 \text{ s}$$

The speed-up of the Numba JIT version relative to the reference implementation is therefore

$$S_{\text{Numba}} = \frac{T_{10,\text{ref}}}{T_{10,\text{Numba}}} = \frac{103.6}{16.36} \approx 6.33$$

This is a large improvement, especially considering that the computation is still performed on a single CPU core. The main gain comes from replacing the masking-heavy NumPy implementation with compiled nested loops, which greatly reduce Python overhead and memory traffic.

2. Function explanation

The JIT version of the Jacobi solver is located in folder `Task_7`, file `simulate_numba.py`.

The function uses two arrays, `u_old` and `u_new`, so that each Jacobi update is computed from the values of the previous iteration. The grid dimensions are read once and used as loop bounds. The main work is done inside the iteration loop. For each sweep, `delta` is reset to zero and the grid is traversed with two nested loops. If a point is marked as interior by `interior_mask`, its new value is computed as the average of its four neighbors, and `delta` is updated with the largest change seen in that iteration. Non-interior points are copied directly so that walls and outside points remain fixed. After each full sweep, the convergence condition `delta < atol` is checked. If it is satisfied, the function returns immediately. Otherwise, `u_old` and `u_new` are swapped, so the new grid becomes the input for the next iteration without copying the whole array.

To make the access pattern cache-friendly, the loops were ordered with rows in the outer loop and columns in the inner loop. Since NumPy uses row-major storage, this means the

inner loop moves through contiguous memory. Compared to the reference version, this also avoids repeated boolean masking and large temporary arrays, which reduces memory traffic.

3. How long would it now take to process all floorplans?

Using the measured average runtime for the Numba JIT implementation on 10 floorplans

$$T_{10,\text{Numba}} = 16.36 \text{ s},$$

we estimate the runtime for all 4571 buildings by assuming approximately linear scaling with the number of floorplans:

$$T_{\text{all}} \approx T_{10,\text{Numba}} \cdot \frac{4571}{10} \approx 16.36 \cdot \frac{4571}{10} = 7470.89 \text{ s} \approx 2.08 \text{ h}$$

Thus, using the Numba JIT CPU implementation, we estimate that processing the full dataset would take approximately 7.47×10^3 s, or about 2.1 hours.

Task 8

1. Solution explanation

The CUDA kernel with Numba solution is located in folder `Task_8`, file `simulate_cuda.py`.

The CUDA version was implemented using a custom Numba kernel that performs exactly one Jacobi iteration per kernel launch. Each GPU thread is responsible for one grid point in the 512×512 interior domain. If the point belongs to the room interior, the thread computes the new value as the average of the four direct neighbors. If the point is not an interior point, the old value is copied unchanged so that walls and outside points remain fixed. The helper function was structured around two device arrays, one holding the values from the previous iteration and one holding the updated values for the current iteration. After each kernel launch, the two arrays are swapped, so the newly computed grid becomes the input for the next iteration. In this way, the Jacobi update rule is respected, since each iteration always reads from the old grid and writes to a separate new grid.

Since thread synchronization across the full grid is not possible inside a single CUDA kernel, the synchronization is handled implicitly between kernel launches. For this reason, the helper function repeatedly launches the kernel for a fixed number of iterations rather than using the early stopping criterion from the CPU reference implementation.

2. How does performance compare to the reference?

We timed the CUDA implementation as a batch job on the `c02613` queue using 1 GPU. For the same subset of $N = 10$ floorplans, we again performed 5 runs in order to obtain a more stable estimate of the wall-clock time. The measured runtimes were

$$9.56,\ 9.50,\ 9.53,\ 9.50,\ 9.65 \qquad \text{(seconds)}$$

with an average of

$$T_{10,\text{CUDA}} = 9.55 \text{ s}$$

For comparison, the corresponding average runtime of the reference implementation in Task 2 was

$$T_{10,\text{ref}} = 103.6 \text{ s}$$

The speed-up of the CUDA version relative to the reference implementation is therefore

$$S_{\text{CUDA}} = \frac{T_{10,\text{ref}}}{T_{10,\text{CUDA}}} = \frac{103.6}{9.55} \approx 10.84$$

So, the CUDA implementation is about $10.8\times$ faster than the reference implementation on this subset. We also note that the measured runtimes are quite close to each other, which suggests that this implementation gives fairly stable timing results across repeated runs.

3. How long would it now take to process all floorplans?

Using the measured average runtime for the CUDA implementation on 10 floorplans

$$T_{10,\text{CUDA}} = 9.55 \text{ s},$$

we estimate the runtime for all 4571 buildings by assuming approximately linear scaling with the number of floorplans:

$$T_{\text{all}} \approx T_{10,\text{CUDA}} \cdot \frac{4571}{10} \approx 9.55 \cdot \frac{4571}{10} = 4365.48 \text{ s} \approx 1.21 \text{ h}$$

Thus, using the CUDA implementation, we estimate that processing the full dataset would take approximately 4.37×10^3 seconds, or about 1.2 hours.

Task 9

1. How does performance compare to the reference?

We timed the CuPy implementation as a batch job on the `c02613` queue using 1 GPU. For the same subset of $N = 10$ floorplans, we again performed 5 runs in order to obtain a more stable estimate of the wall-clock time. The measured runtimes were

$$22.56,\ 21.63,\ 21.94,\ 22.17,\ 20.78 \qquad \text{(seconds)}$$

with an average of

$$T_{10,\text{CuPy}} = 21.81 \text{ s}$$

The speed-up of the CuPy version relative to the reference implementation is therefore

$$S_{\text{CuPy}} = \frac{T_{10,\text{ref}}}{T_{10,\text{CuPy}}} = \frac{103.6}{21.81} \approx 4.75$$

Thus, the CuPy implementation is about 4.8× faster than the reference implementation on this subset. The improvement is still significant, but it is clearly smaller than for the custom CUDA kernel from the previous task.

2. How long would it now take to process all floorplans?

$$T_{\text{all}} \approx T_{10,\text{CuPy}} \cdot \frac{4571}{10} \approx 21.81 \cdot \frac{4571}{10} = 9969.95 \text{ s} \approx 2.77 \text{ h}$$

Thus, using the CuPy implementation, we estimate that processing the full dataset would take approximately 9.97×10^3 seconds, or about 2.8 hours.

3. Was anything surprising about the performance?

Yes. The main surprising result was that the CuPy version was slower than the custom CUDA implementation, even though both run on the GPU. The reason is likely that the CuPy solver keeps the same masking-heavy structure as the original NumPy code, including repeated temporary arrays and convergence checks, while the custom CUDA version uses a simpler and more direct kernel-based update. So, although CuPy was easier to implement, it did not use the GPU as efficiently as the custom CUDA solution.

Task 10

Profiling the CuPy implementation with `nsys` showed that the main performance issue was repeated synchronization between the GPU and the CPU, together with a large number of small kernel launches. In the CUDA API summary, `cuLaunchKernel` was called 935 times, while `cudaStreamSynchronize` was called 205 times. In addition, `cudaMemcpyAsync` was called 187 times. The GPU memory summary further showed 175 device-to-host copies, even though the total transferred size was very small. This strongly suggests that small values were repeatedly transferred back to the CPU during execution.

This matches the structure of the CuPy solver, where the convergence check

```
delta = cp.abs(...).max()
if delta < atol:
    break
```

requires the scalar value of `delta` to be available on the CPU, forcing synchronization in every iteration. At the same time, the GPU kernel summary showed many small kernels such as `cupy_getitem_mask`, `cupy_subtract`, `cupy_absolute`, and reduction-related kernels, instead of one large kernel performing the full update. Thus, the CuPy implementation spends a noticeable part of its time on launch overhead and synchronization rather than only on the Jacobi update itself.

Solution

The solution is located in folder `Task_10`, file `simulate_cupy_fixed.py`.

The earlier CuPy implementations were based on high-level array expressions. First, we translated the NumPy reference solver directly to CuPy. This moved the work to the GPU, but each Jacobi iteration was still split into many separate operations such as additions, masking, convergence checking, and masked assignment. Profiling showed that this led to many small kernel launches and repeated synchronization overhead. We then tried removing the convergence check and running a fixed number of iterations, but this still relied on high-level CuPy operations and did not solve the main problem.

The real issue was that one logical Jacobi iteration was being expressed as many small GPU kernels. The only proper fix was therefore to write the Jacobi update as a single custom GPU kernel. This was done using `cp.RawKernel`, where the kernel body is written explicitly in CUDA C.

In the raw kernel, each GPU thread handles one interior grid point. The thread computes its coordinates, checks whether the point belongs to the room interior, and if so computes the new value as the average of the four neighboring points from the old grid. Otherwise, the old value is copied unchanged. The helper function stores the old and new grids as two CuPy arrays on the GPU and launches the kernel once per Jacobi iteration, swapping the two array references after each launch.

This is much faster because one kernel launch now performs one full Jacobi step, instead of splitting the work into many small array operations. As a result, the kernel-launch

overhead is greatly reduced and the GPU is used more efficiently. For $N = 10$ floorplans, this implementation ran in

$$T_{10,\text{CuPyRaw}} = 2.33 \text{ s},$$

which is substantially faster than the earlier CuPy versions and also faster than the previous custom CUDA implementation.

Using this measured runtime, the estimated runtime for all 4571 buildings is

$$T_{\text{all}} \approx T_{10,\text{CuPyRaw}} \cdot \frac{4571}{10} \approx 2.33 \cdot \frac{4571}{10} = 1067.96 \text{ s} \approx 17.78 \text{ min}$$

The results remained numerically very close to those of the reference implementation. Small deviations of about 0.005 were considered acceptable, since they are consistent with normal floating-point differences and do not change the conclusions of the analysis.

Task 12

For the final full-dataset run, we used the optimized GPU implementation developed at the end of Task 10. This version uses a `CuPy RawKernel`, where one custom GPU kernel performs one full Jacobi iteration over the grid. Each GPU thread updates one interior grid point directly, which avoids the large kernel-launch overhead seen in the earlier high-level CuPy versions.

The simulation was run on the `c02613` cluster and took approximately 11.5 minutes to complete. The script was run on all 4571 floorplans, and the output was written to a CSV file for later analysis with Pandas. To keep the CSV file clean, the total runtime was printed to the error stream instead of standard output, so that only the header and the per-building statistics were written to the CSV file.

1. What is the distribution of the mean temperatures?

The distribution of the mean temperatures is shown in Fig. 6. The histogram is centered roughly around 15–16°C, with most buildings concentrated in that range and fewer buildings at the lower and higher ends. This suggests that, for a large part of the dataset, the average indoor temperature remains below a comfortable room temperature.

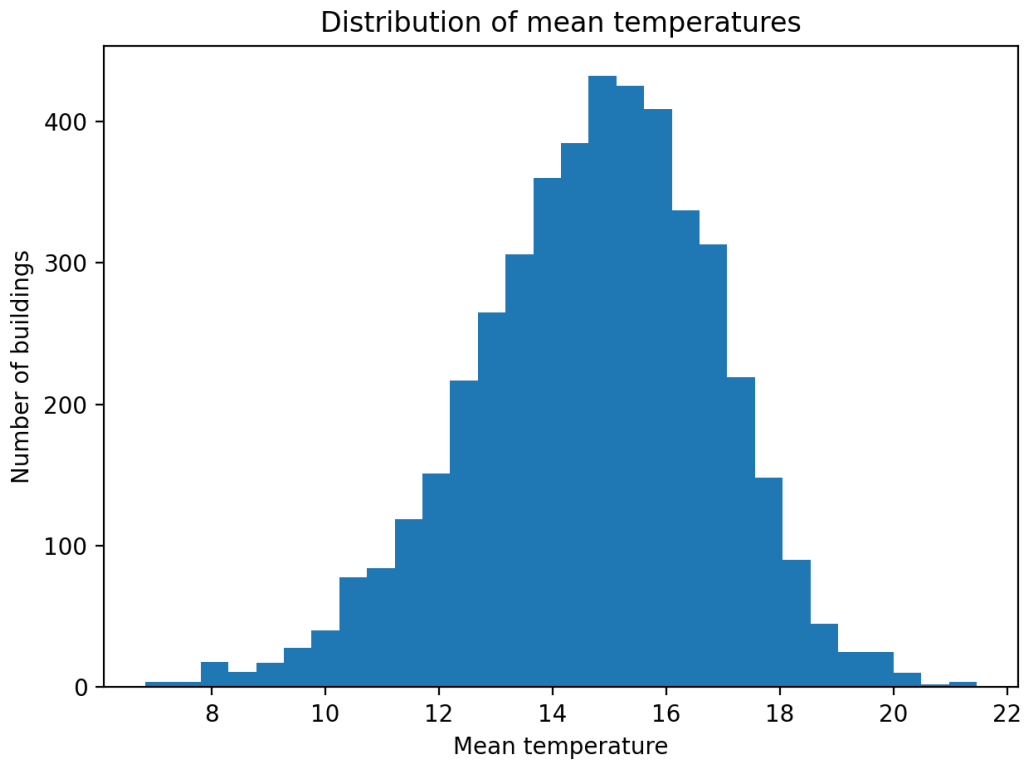

Figure 6: Distribution of the mean temperatures across all 4571 buildings.

2. What is the average mean temperature of the buildings?

The average mean temperature across all 4571 buildings was

$$14.71°\text{C}$$

3. What is the average temperature standard deviation?

The average temperature standard deviation across all buildings was

$$6.80°\text{C}$$

This indicates that the temperature variation inside the buildings is, on average, fairly large.

4. How many buildings had at least 50% of their area above 18C?

A total of

$$813$$

buildings had at least 50% of their interior area above 18°C.

5. How many buildings had at least 50% of their area below 15C?

A total of

$$2461$$

buildings had at least 50% of their interior area below 15°C.